

Package Inspection App

Initial UI/UX plan, mockups, and implementation guide

Role: UI/UX Developer | Project: AI-based package inspection (Flutter app, FastAPI backend, YOLO model) | Status: design spec for review

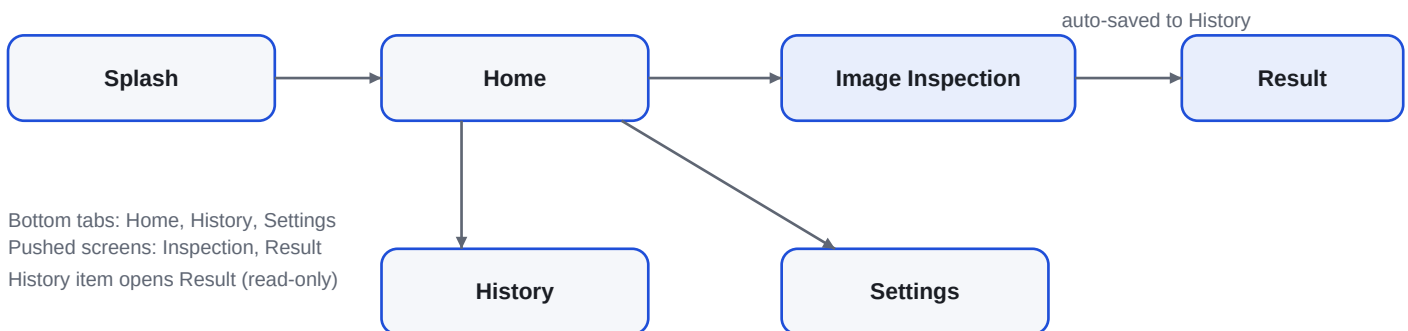
1. Purpose and scope

This document defines the first mobile UI for the package inspection app: six screens, the main user flow, how detections are shown, and what the Flutter developers need to build it. It is a design specification only. No Flutter code is included, and no backend or model decisions are made here; those are listed as dependencies for the owning roles.

Design goals

- **Lightweight:** three tabs, two pushed screens, no extra features.
- **Camera-first:** the most common task (photograph a package, get a verdict) takes three taps and is thumb-reachable.
- **Honest AI output:** confidence is always visible, low-confidence detections look different, and the app never claims more certainty than the model returns.
- **Consistent:** one set of tokens and six reusable components across all screens.

2. Navigation and user flow



Primary flow: Home, then **Inspect package**, capture or pick a photo, review, **Inspect**, result. Every result is saved to History automatically unless turned off in Settings. Back always moves one step; from Result it returns to Home.

Out of scope for v1

Dark mode, user accounts, share or export, search and filters, analytics dashboard, localization, onboarding tutorial, and live video overlay. Whether live detection belongs in v1 is an open question for the Project Lead.

3. Design system

Colors

	Token	Hex	Use
	primary	#1F4FD8	Buttons, active tab, links
	textPrimary	#1A1D23	Headings and body
	textSecondary	#5F6673	Captions, hints
	background	#F5F7FA	Screen background
	divider	#E2E6EC	Lines, card borders
	pass	#1E8E3E	No defects
	defect	#D93025	Defect boxes and verdict
	review	#B26A00 / #F9AB00	Low confidence (text / fill)
	cameraBg	#000000	Camera and image viewer

Light theme only for v1. Status is never conveyed by color alone: every state also has an icon and a text label.

Typography (system font: Roboto on Android, SF on iOS, no custom font files)

Style	Size / weight	Use
Title	24 / Bold	Screen titles, verdict
Heading	18 / SemiBold	Section headers, list titles
Body	16 / Regular	Main text
Caption	13 / Regular	Timestamps, hints, metadata
Button	16 / SemiBold	All buttons

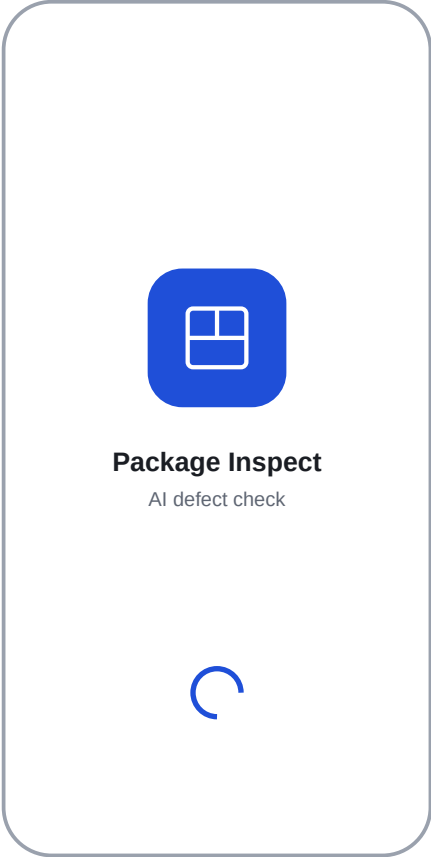
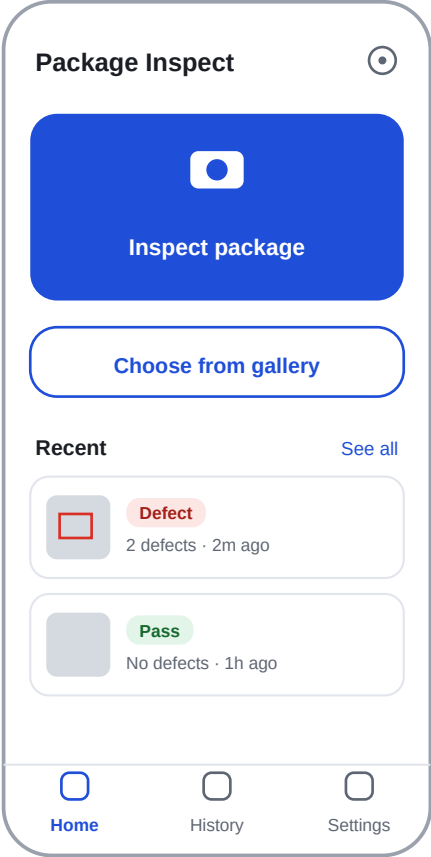
Spacing, shape, and touch

- 4 dp grid: 4, 8, 16, 24, 32. Screen side padding 16. Gap between cards 12.
- Corner radius: cards 12, buttons 12, chips 8. Flat cards with a 1 dp divider border, no heavy shadows.
- Minimum touch target 48 x 48 dp. Button height 52. Text must stay usable at 130% system text scale.

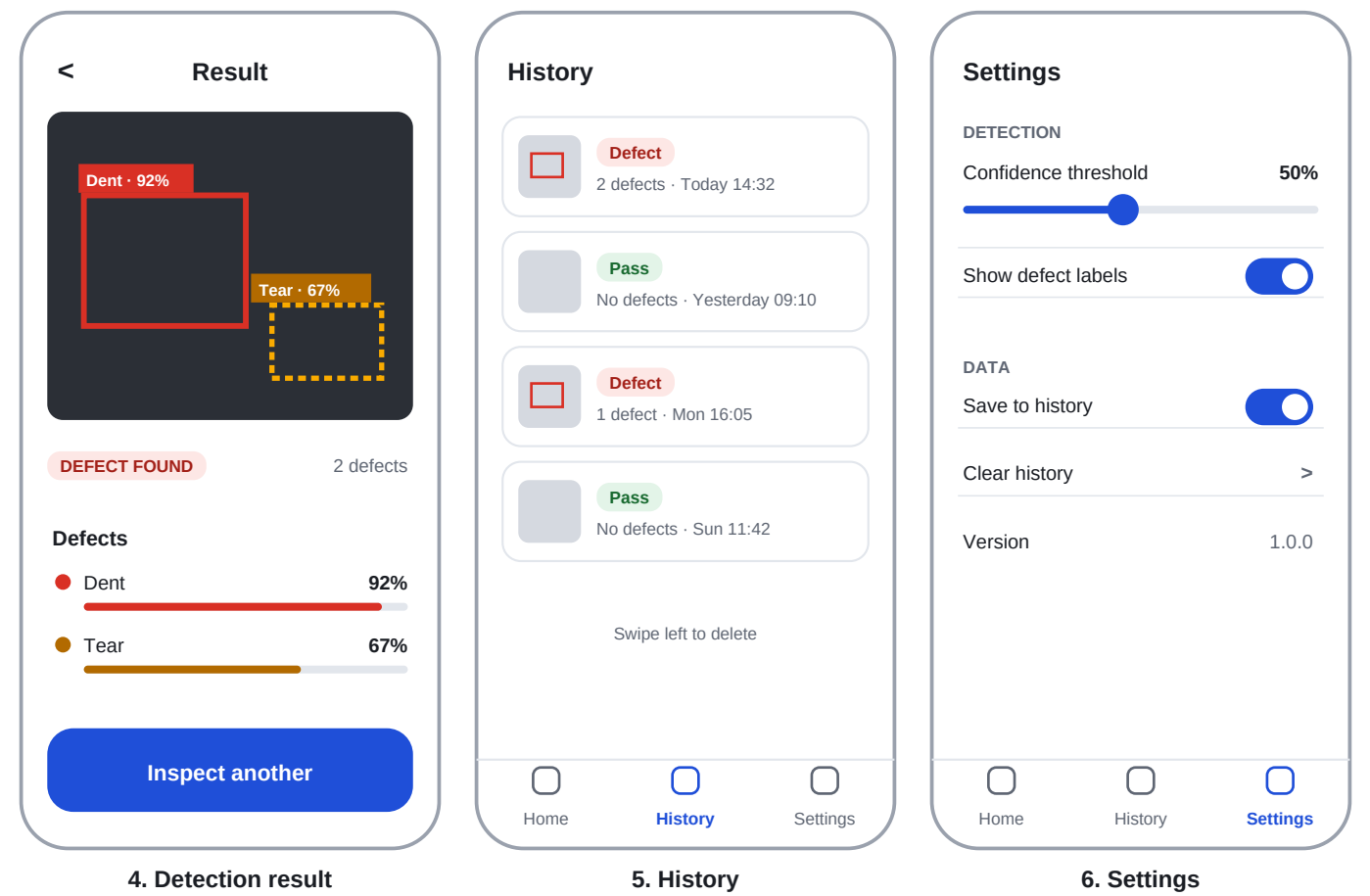
Core components (build once, reuse everywhere)

Component	Used on	Notes
PrimaryButton / SecondaryButton / TextButton	All screens	One primary button per screen
StatusBadge (PASS, DEFECT, REVIEW)	Home, Result, History	Icon + text + color
DefectChip (class + percent)	Result image, defect list	Example: Dent 92%
InspectionCard	Home recent list, History	Thumbnail, verdict, time
StateView	Every screen	One widget for loading, empty, error
AppBar	Pushed screens	Back arrow + title

4. Screens 1 to 3

		
1. Splash	2. Home	3. Image inspection
Layout: centered logo, app name, small spinner. Actions: none. Auto-continues to Home as soon as init finishes (max about 1.5 s, no forced delay). Error: init failure shows StateView with Retry.	Layout: large Inspect package card, Choose from gallery button, last 3 inspections, bottom tabs. Actions: Inspect package opens camera; gallery opens picker; recent card opens Result; See all opens History. Empty: "No inspections yet" with a prompt to take a photo.	Capture mode: full-width preview, corner guide frame, one-line hint, flash toggle, gallery shortcut, shutter at bottom center. After capture the same screen shows Review: photo, Retake, Inspect. Permission: explain why camera is needed; Allow Camera and Choose from Gallery; if permanently denied, Open Settings. Loading: dimmed image, spinner, "Inspecting package...", Cancel.

5. Screens 4 to 6



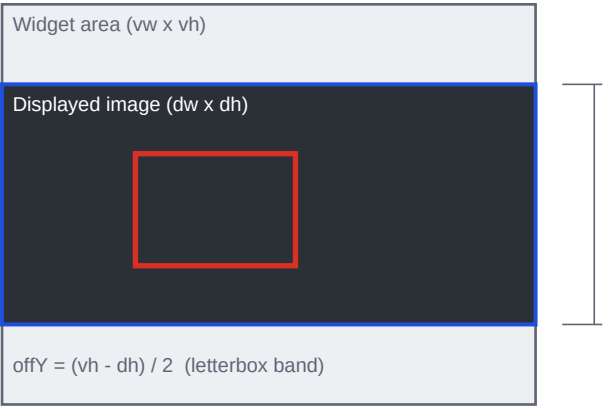
Layout: image with boxes, verdict badge and count, defect list, Inspect another. Interaction: pinch/pan to zoom; tap a box or list row to select both; back returns to Home. Verdicts: PASS (green), DEFECT FOUND (red), REVIEW (amber, low confidence).	Layout: reverse-chronological InspectionCards. Actions: tap opens Result read-only; swipe left deletes with Undo. States: skeleton cards while loading; empty message with Inspect package button; error with Retry. No filters in v1.	Controls: confidence threshold slider (10-95%, step 5, default 50% proposed), defect labels toggle, save-to-history toggle, Clear history, version. Clear history: confirm dialog "Delete all inspection history? This can't be undone." Settings apply instantly, no Save button.
--	--	--

6. Presenting detections

Element	Specification
Bounding box	3 dp outline, defect red. Low-confidence boxes are dashed amber so they differ without relying on color. Selected box gets a 4 dp stroke and others dim to 40%.
Label chip	"Class - 92%", 13 sp semibold, white text on box color. Sits above the box top-left, moves inside if near the top edge. With more than 3 boxes, chips show only for the selected box.
Confidence	Always an integer percent (92%, never 0.9234). Defect list rows also show a thin progress bar. Proposed bands, pending AI/ML: 75% and up high, 50-74% medium, under 50% low (amber, "Low confidence").
Defect list row	Color dot, class name, percent and bar. Tapping a row highlights its box, and tapping a box scrolls to its row.
Accessibility	Icon + text + color for every state. Screen-reader text per row, for example "Dent, 92 percent confidence".

Box-to-image mapping (the most common implementation bug)

The photo is shown with contain-fit, so it is letterboxed inside the widget. Boxes must be mapped into the **displayed image rectangle**, not the widget rectangle. Use these steps in whatever painter or overlay approach the Flutter team prefers:



1. $scale = \min(vw / iw, vh / ih)$
2. $dw = iw * scale$; $dh = ih * scale$
3. $offX = (vw - dw) / 2$; $offY = (vh - dh) / 2$
4. Normalized box: $x = offX + nx * dw$; $y = offY + ny * dh$
Pixel box: $x = offX + px * scale$; $y = offY + py * scale$
5. Clamp the box to the displayed image rect
6. Label chip above the box; if no room, inside the top edge
7. Overlap: tap selects the smallest-area box under the finger

7. Loading, empty, error, and success states

Screen	Loading	Empty	Error	Success
Splash	Spinner	n/a	"Something went wrong" + Retry	Auto-go to Home
Home	Skeleton recent cards	"No inspections yet"	Hide recent section, show Retry link	Recent list
Image Inspection	"Inspecting package..." overlay + Cancel	n/a	See table below	Navigate to Result
Result	n/a (data in hand)	PASS: "No defects found"	Image fails: placeholder, list still shown	Boxes + list
History	3 skeleton cards	"No inspection history yet"	"Couldn't load history" + Retry	List
Settings	n/a	n/a	Snackbar if save fails	Applies instantly

Inspection error messages (stay on Review, keep the image)

Case	Message	Actions
No internet	No internet connection. Check your network and try again.	Try Again, Cancel
Server unreachable	Can't reach the inspection server.	Try Again, Cancel
Timeout (30 s proposed)	Inspection is taking too long.	Try Again, Cancel
Invalid image	This image can't be used. Try another photo.	Retake, Gallery
Server error	Something went wrong on our side. Try again.	Try Again, Cancel

Success feedback: a light haptic and a direct jump to Result. No toast needed.

Copy rules

- Sentence case everywhere. Button labels start with a verb and are 1-3 words.
- Error copy says what happened, then what to do. No raw exception text, no "Error:" prefix.
- Avoid "please" and "successfully". Use contractions.

8. Implementation guide for Flutter developers

Adapt this to the existing Flutter architecture. It describes structure and behavior, not code.

Suggested structure

Area	Contents
Theme / tokens	One tokens file (colors, spacing, radii, text styles) feeding the app theme. Screens never hard-code values.
Shared widgets	The six core components. Every screen state comes from StateView.
Features	splash, home, inspection (capture, review, result), history, settings. Each with its own state management (BLoC if that is the project standard).
Data	Inspection and Detection models, a repository for the detection API, a local store for history and settings.

Routes

Route	Screen	Notes
/splash	Splash	Replaces itself with /home
/home	Tab shell: Home, History, Settings	Bottom navigation lives here only
/inspect	Image Inspection	Pushed, capture and review modes
/result	Detection Result	Takes an inspection; flag readOnly when opened from History

Inspection screen state machine

idle (permission check) -> capturing -> reviewing -> uploading -> success (go to Result) or failure (back to reviewing with an error and the image kept). Cancel during uploading returns to reviewing and aborts the request.

Data the UI needs

- **Inspection:** id, local image path, created time, overall status, list of detections, original image width and height.
- **Detection:** class label, confidence (0 to 1), bounding box. Box format must be confirmed by Backend (normalized or pixels, corner or center format).
- Display-side filtering by the Settings threshold; do not rewrite the stored result.

Performance and quality

- Resize and compress the photo before upload (long side about 1280 px, to be agreed with Backend).
- Generate small thumbnails for lists. Use lazy lists and skeletons, not blocking spinners.
- Repaint the overlay only when the result, selection, or zoom changes.
- Handle camera permission states explicitly, including permanently denied.
- Respect system text scale up to 130% and add screen-reader labels to boxes and rows.

9. Suggested build order

Step	Deliverable	Needs
1	Tokens, theme, six core components	Nothing
2	App shell, routes, bottom tabs with static data	Step 1
3	Capture and review flow, permissions	Step 2
4	Result screen and box overlay using a mock JSON result	Step 2, box format decision
5	History and Settings with local persistence	Step 2
6	Wire the real detection API and error states	Backend contract

10. Dependencies and open questions

Owner	Needed from them
Backend	Response fields: overall status (or the rule to derive it), per-detection label, confidence, box. Box format, image size used for inference, error codes (invalid image, server error, timeout), max upload size, expected timeout.
AI/ML	Final class names and display names, recommended default confidence threshold, the review band, typical inference time.
Project Lead	Is live video detection in v1? Is history on-device only or synced? Is authentication in v1?
QA	Use the acceptance checklist below as the starting test plan.

11. Acceptance checklist

- All six screens match the mockups and use only token values.
- Loading, empty, error, and success states are reachable on every screen where they apply.
- Touch targets are at least 48 dp; text stays usable at 130% scale.
- Boxes align with the photo for portrait, landscape, and non-camera aspect ratios.
- Text contrast is at least 4.5:1; status is never color-only.
- Cancel works during inspection; Retake and Try Again keep the chosen image.
- History survives app restart; Clear history asks for confirmation.

Note: mockups are low-fidelity layout references, not final visual design. Class names, thresholds, and timeouts shown are placeholders until confirmed by AI/ML and Backend.